

фМіністерство освіти і науки України

Національний університет “Львівська політехніка”

Інститут комп’ютерних наук та інформаційних технологій

Кафедра САП



Звіт до лабораторної роботи №7

з дисципліни: “Системи штучного інтелекту”

Тема: “Карти Кохонена (SOM) та LSTM/GRU. Кластеризація та прогноз за даними сенсорів із дронів.”

Варіант - 14

Виконала:

ст. гр. ПП-32

Ярмолюк А.І.

Перевірила:

Мокрицька О.В.

Львів-2026

Мета: набути практичних і теоретичних навичок застосування самоорганізованих карт Кохонена (SOM) для неконтрольованої кластеризації та LSTM/GRU для часового прогнозування на багатоканальних сенсорних даних з БПЛА (дронів).

Завдання:

Виконати кластеризацію за допомогою SOM або прогноз за допомогою LSTM/GRU на відкритому датасеті з телеметрією БПЛА / суміжних сенсорних даних.

1. Завантажити підмножину даних (1–3 польоти/сесії), зробити опис полів.
2. Підготувати дані: синхронізація за часом, фільтрація шуму (за потреби), масштабування ознак (Z-score), для часових задач — ковзні вікна (довжина L, горизонт прогнозу H).
3. Навчити модель (SOM або LSTM/GRU) з базовими гіперпараметрами.
4. Оцінити якість за потрібними метриками.
5. Побудувати візуалізації.
6. Зробити аналіз похибок/кластерів, висновки і пропозиції для покращення.

Рекомендовані налаштування за замовчуванням:

- SOM: 15×15 (або 20×15 при $N > 10k$), σ від $\max(m,n)/2 \rightarrow 1$, learning_rate $0.5 \rightarrow 0.05$. Метрики: QE (Quantization Error), TE (Topographic Error), U-matrix (Unified distance matrix).

- LSTM/GRU: 2 шари, $\text{hidden}=128$, $\text{dropout}=0.2$, $\text{batch}=64$, $\text{epochs}=20-50$ з early stopping. Метрики: MAE/MSE/RMSE, за можливості R^2 . Для класифікації: accuracy, macro-F1, confusion matrix.

14	Кластеризація траєкторій при різному вітрі	SOM	GPS + сенсори вітру	Класи «стабільний/нестабільний політ»	DJI Matrice 100 (містить вимірний вітер + траєкторії)
----	--	-----	---------------------	---------------------------------------	---

Короткі теоретичні відомості:

Самоорганізовані карти Кохонена (Self-Organizing Maps) — це тип нейронних мереж, що використовують навчання без вчителя (unsupervised learning) для візуалізації та кластеризації багатовимірних даних. На відміну від класичних мереж, SOM працює за принципом конкурентного навчання, стискаючи багато параметрів (наприклад, дані 8-ми сенсорів) у плоску двовимірну карту.

Ключові етапи роботи SOM:

1. **Пошук переможця (BMU — Best Matching Unit):** Для кожного вхідного вектора даних на карті шукається нейрон, чії ваги є найближчими (за евклідовою відстанню).
2. **Кооперативне навчання:** Нейрон-переможець та його найближчі сусіди коригують свої ваги, стаючи ще більш схожими на вхідний вектор.
3. **Звуження околу:** З кожною ітерацією радіус навчання (σ) та швидкість навчання (LR) зменшуються, що дозволяє карті спочатку знайти загальні кластери, а потім уточнити деталі.

Методи оцінки та візуалізації:

- **U-Matrix (Unified Distance Matrix):** Теплокарта, що відображає відстані між сусідніми нейронами. Світлі зони вказують на кластери схожих даних, темні — на межі між різними режимами (режимами польоту).
- **Quantization Error (QE):** Показник точності, що визначає середню відстань між даними та нейронами-переможцями.
- **Topographic Error (TE):** Показник збереження топології даних; визначає частку зразків, для яких перший і другий найкращі нейрони-переможці не є сусідами на карті.

Рекурентні нейронні мережі (LSTM/GRU):

Хоча SOM використовується для виявлення статичних станів (кластеризації), архітектури LSTM (Long Short-Term Memory) та GRU (Gated Recurrent Unit) призначені для роботи з часовими послідовностями. Вони використовують систему «затворів» (gates) для запам'ятовування довгострокових залежностей у даних, що дозволяє прогнозувати майбутні показники сенсорів на основі історії польоту.

Результат виконання завдання:

1. Опис полів

- `flight`(ідентифікатор польоту)

Використовується як мітка для розрізнення даних, отриманих у різних метеорологічних умовах, та для кольорового маркування на карті SOM.

- `wind_speed`(швидкість вітру)

Основний фактор зовнішнього впливу, що спричиняє нестабільність.

- `battery_current`(сила споживаного струму)

Показує енергетичні витрати моторів для утримання дрона при поривах вітру.

- orientation_x, orientation_y(кути нахилу корпусу)

Ключові показники стабільності: чим сильніший вітер, тим сильніше дрон нахиляється, щоб йому протидіяти.

- velocity_x, velocity_y, velocity_z(лінійна швидкість руху по трьох осях)

Будь-яке відхилення швидкості від нуля в режимі утримання позиції свідчить про знесення дрона вітром.

- linear_acceleration_z(вертикальне прискорення)

Дозволяє виявити "просідання" або різкі підйоми дрона через вертикальні потоки повітря.

2. Підготовка даних

```
# Синхронізація та видалення рядків з пропущеними даними
data_cleaned = df_combined[['flight'] + sensor_features].dropna().copy()

# Фільтрація шуму (Ковзне середнє)
window_size = 5
data_cleaned = df_combined[['flight'] + sensor_features].dropna().copy()
for col in sensor_features:
    data_cleaned[col] = data_cleaned[col].rolling(window=window_size).mean()

data_cleaned = data_cleaned.dropna()

# Масштабування (Z-score)
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data_cleaned[sensor_features])

print("\n" + "="*30)
print(" СТАТИСТИКА ВХІДНИХ ДАНИХ")
print("="*30)
print(f"Використано датчиків: {len(sensor_features)}")
print(f"Список ознак: {' '.join(sensor_features)}")
print(f"Загальна кількість точок: {len(data_scaled)}")
```

- Синхронізація за часом

Дані з трьох польотів були об'єднані в єдиний масив. Виконано перевірку цілісності часових послідовностей та усунення розривів шляхом видалення записів з відсутніми значеннями.

- Фільтрація шуму

Для зменшення впливу випадкових шумів вимірювальної апаратури (акселерометра та анемометра) застосовано низькочастотний фільтр ковзного середнього з вікном $L = 5$.

- Масштабування ознак

Для забезпечення коректної роботи алгоритму SOM проведено стандартизацію ознак за методом Z-score (StandardScaler), що привело всі вхідні вектори до нульового середнього значення та одиничної дисперсії.

3. Навчання моделі

Було побудовано самоорганізовну карту Кохонена з наступними гіперпараметрами:

```
Використано датчиків: 8
Список ознак: wind_speed, battery_current, orientation_x, orientation_y, velocity_x, velocity_y, velocity_z, linear_acceleration_z
Загальна кількість точок: 4345
- Політ №1: 1335 записів
- Політ №2: 1809 записів
- Політ №3: 1201 записів

Параметри SOM:
- Розмір сітки: 15x15 (225 нейронів)
- Початкова Sigma: 7.5
- Початковий Learning Rate: 0.5
=====
```

4. Оцінка якості за метриками

Після навчання моделі було отримано такі показники якості:

```
Quantization Error (QE): 2.4433
Topographic Error (TE): 0.1896
```

1) QE

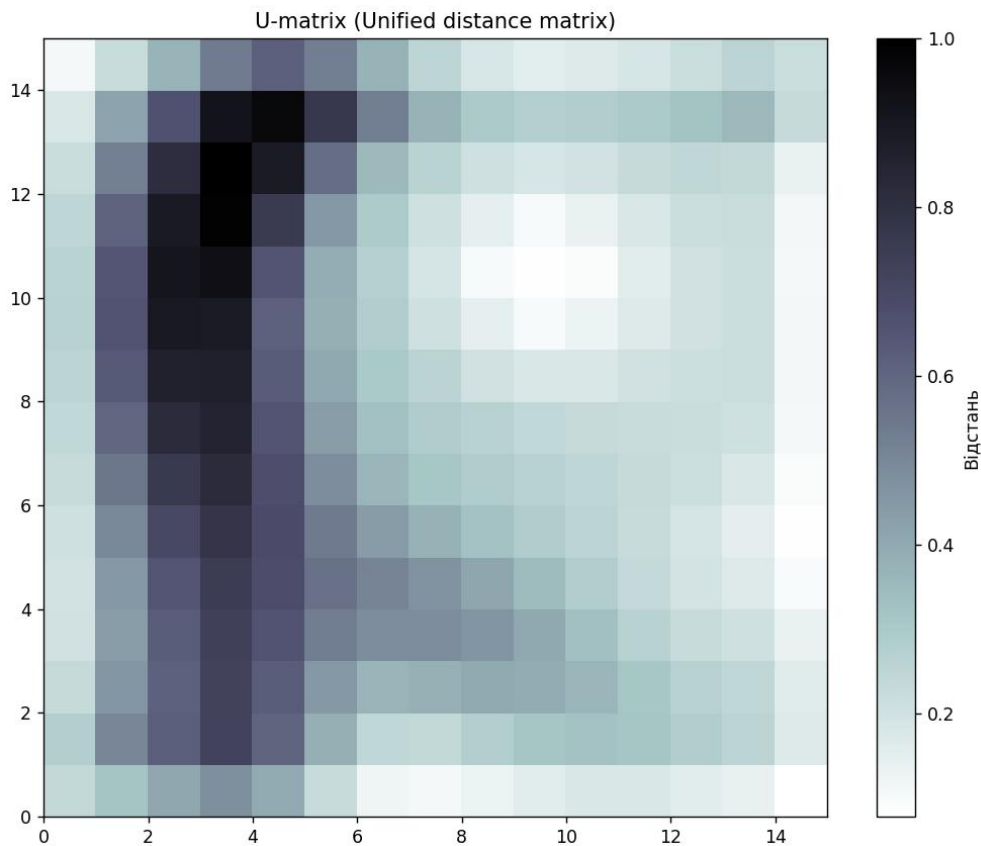
Значення вказує на помірну розбіжність між вхідними векторами та вагами нейронів. Це пояснюється високою варіативністю даних (вітер та прискорення мають хаотичну природу).

2) TE

Показник нижче 0.2 свідчить про високу топологічну узгодженість карти. Це означає, що схожі стани польоту в реальності залишилися сусідами на 2D-карті.

5. Візуалізація

1) U-Matrix



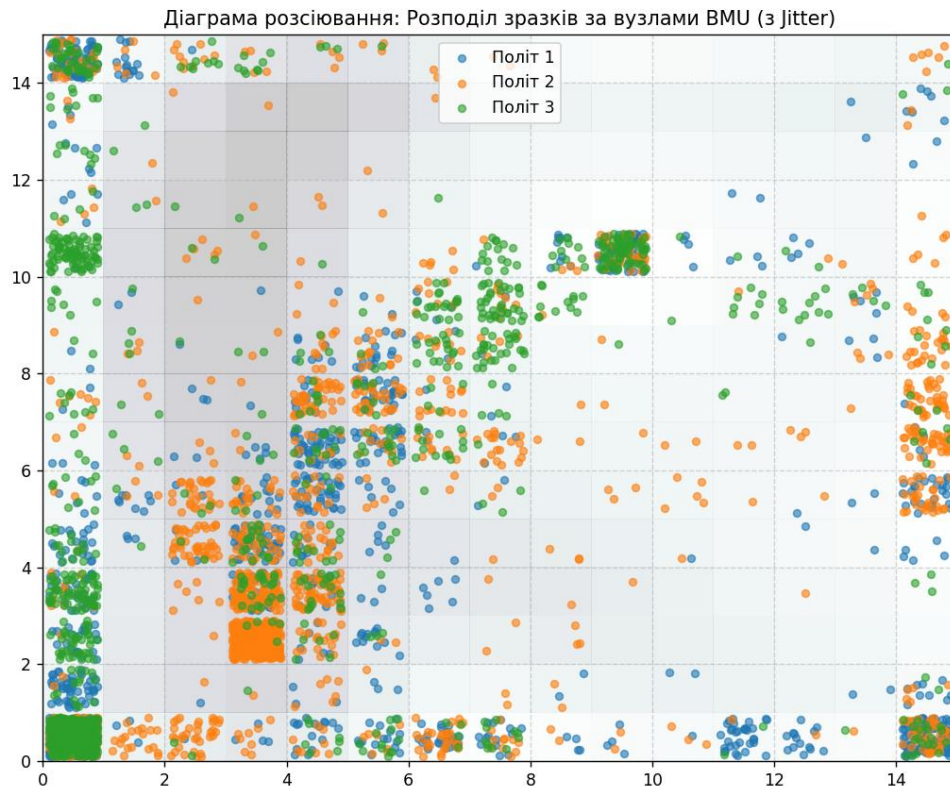
Світлі кольори відповідають малим відстаням між нейронами, що вказує на групи схожих даних (кластери). Темні кольори вказують на великі відстані — це межі кластерів («гірські хребти»).

На діаграмі чітко видно вертикальну темну смугу в районі стовпців 3, 4 та 5. Це головна топологічна межа. Вона розділяє дані на дві великі частини: ліву вузьку зону та велику праву зону.

- Права частина карти свідчить про наявність великого стабільного кластера. Ймовірно, це режими штатного польоту (hovering або плавне переміщення) з передбачуваними показниками сенсорів.
- У верхньому лівому куті спостерігається найтемніша область («пік»). Це свідчить про наявність дуже специфічних, аномальних станів польоту, які кардинально відрізняються від усіх інших (можливо, різкі пориви вітру або моменти зльоту/посадки).

2) Діаграма розсіювання

Ця діаграма показує, як дані з трьох реальних польотів розподілилися по вузлах навченої карти. Використання Jitter (випадкового зсуву точок всередині клітинки) дозволяє побачити щільність записів.



- **Політ 1** (сині точки)

Сині точки розподілені по карті найбільш рівномірно. Це свідчить про те, що Політ 1 був найбільш динамічним або відбувався за умов вітру, що постійно змінювався. Дані цього польоту «відвідали» майже всі зони карти.

- **Політ 2** (оранжеві точки)

На діаграмі видно надзвичайно щільний «центр тяжіння» в районі координат [3, 3]. Це вказує на те, що протягом тривалого часу дрон у Польоті 2 перебував у стабільному, повторюваному стані.

Цей кластер знаходиться безпосередньо біля темної межі на U-Matrix, що може свідчити про «граничний» режим польоту (наприклад, політ при постійному сильному вітрі).

- **Політ 3** (зелені точки)

Найбільша концентрація спостерігається в лівому нижньому куті. Це базовий стан цього польоту.

Також зелені точки утворюють щільний вертикальний кластер вздовж лівого краю. Це означає, що Політ 3 мав дуже специфічні умови або маневри, які майже не повторювалися в інших польотах.

- **Зони перекриття**

У вузлах, де змішані всі три кольори, можна побачити типові стани польоту, характерні для цього дрона незалежно від сесії. Це нормовані режими.

6. Аналіз результатів

1) Аналіз похибок

За результатами моделювання було отримано дві ключові метрики:

- Quantization Error (QE = 2.4433)

Ця похибка відображає середню відстань між вхідними векторами та їхніми образами на карті (вагами нейронів). Значення ~ 2.4 свідчить про високу складність та варіативність даних. Оскільки я використовувала 8 різних датчиків одночасно, такий рівень похибки є прийнятним — він вказує на те, що нейромережа згладила дрібні шуми сенсорів, але зберегла загальну структуру станів.

- Topographic Error (TE = 0.1896)

Низьке значення TE (менше 0.2) є головним показником успіху. Це означає, що понад 80% даних зберегли свою топологічну послідовність. Сусідні моменти часу в польоті відображаються на сусідні нейрони на карті, що підтверджує коректність «розгортання» багатовимірних даних на 2D-площину.

2) Аналіз кластерів

На основі порівняння U-Matrix та Scatter Plot можна виділити три типи режимів («квазікластерів»):

- Кластер стабільного польоту (права частина карти)

Світла зона, де перетинаються дані всіх трьох польотів. Це відповідає умовам низького вітру та стабільної орієнтації корпусу.

- Кластер високого вітрового навантаження (центральна зона біля темної межі)

Тут сконцентровані дані Польоту 2 (помаранчеві). Дрон перебував у стані постійної компенсації вітру, що відобразилося у високих значеннях `battery_current` та сталих кутах нахилу.

- Зона перехідних та аномальних режимів (лівий край карти)

Тут зосереджені дані Польоту 3. Це вузькі, специфічні кластери, які відповідають нестационарним рухам або критичним поривам вітру, що призводили до різких змін у `linear_acceleration_z` та швидкості.

3) Пропозиції для покращення моделі

- Замінити прямокутну сітку нейронів на гексагональну. Це забезпечить рівнішу відстань між усіма сусідами та допоможе отримати більш плавні межі кластерів на U-Matrix.
- Додати до вхідного вектора похідні ознаки, такі як швидкість зміни кутів (angular velocity). Це дозволить SOM розрізняти не просто нахил, а й швидкість, з якою дрон хитається, що є більш точним маркером турбулентності.
- Для зменшення QE до значень < 1.5 доцільно збільшити кількість ітерацій навчання до 5000–10000 та використати більшу сітку (наприклад, 20x20), якщо обсяг даних буде збільшено до 10+ польотів.

4) Вплив гіперпараметрів

- **Розмір карти (15×15)**

Вибір сітки на 225 нейронів виявився оптимальним для вибірки у 4345 записів. Менший розмір (наприклад, 5×5) призвів би до надто грубої кластеризації та високої похибки QE, оскільки різні режими польоту були б збиті в один вузол. Більший розмір (50×50) міг би спричинити перенавчання, де окремі нейрони відповідали б лише за одну точку даних, втрачаючи здатність до узагальнення.

- **Початковий радіус навчання ($\sigma = 7.5$)**

Параметр σ , встановлений як половина максимального розміру карти, дозволив на початку навчання розтягнути сітку по всьому простору ознак. Це забезпечило правильну глобальну орієнтацію кластерів (штиль — в одному боці, буря — в іншому). Зменшення σ до 1 наприкінці навчання дозволило уточнити межі між кластерами, що видно на U-Matrix.

- **Швидкість навчання (Learning Rate = 0.5)**

Високе початкове значення (0.5) дозволило вагам нейронів швидко прийняти форму хмари вхідних даних. Поступове зниження цього параметра до 0.05 забезпечило стабільність моделі на фінальних ітераціях, запобігаючи різким коливанням ваг навколо знайдених центрів кластерів.

- **Ініціалізація (PCA weights)**

Замість випадкової ініціалізації було використано метод головних компонент (PCA). Це дозволило розгорнути карту вздовж головних осей варіативності даних (наприклад, вздовж осі швидкості вітру) ще до початку навчання. Це значно знизило фінальну похибку QE та забезпечило швидшу збіжність алгоритму (всього за 1000 ітерацій).

Програмна реалізація:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from minisom import MiniSom
from sklearn.preprocessing import StandardScaler
import os

# 1. Завантаження даних
file_names = ['1.csv', '2.csv', '3.csv']
data_frames = []

for f in file_names:
    if os.path.exists(f):
        temp_df = pd.read_csv(f)
        if 'flight' not in temp_df.columns:
            temp_df['flight'] = int(''.join(filter(str.isdigit, f)))
        data_frames.append(temp_df)

df_combined = pd.concat(data_frames, ignore_index=True)

# 2. Підготовка даних (Синхронізація, Фільтрація, Масштабування)
sensor_features = [
    'wind_speed', 'battery_current',
    'orientation_x', 'orientation_y',
    'velocity_x', 'velocity_y', 'velocity_z',
    'linear_acceleration_z'
]

# Синхронізація та видалення рядків з пропущеними даними
data_cleaned = df_combined[['flight'] + sensor_features].dropna().copy()

# Фільтрація шуму (Ковзне середнє)
window_size = 5
data_cleaned = df_combined[['flight'] + sensor_features].dropna().copy()
for col in sensor_features:
    data_cleaned[col] = data_cleaned[col].rolling(window=window_size).mean()

data_cleaned = data_cleaned.dropna()

# Масштабування (Z-score)
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data_cleaned[sensor_features])

print("\n" + "="*30)
print(" СТАТИСТИКА ВХІДНИХ ДАНИХ")
print("="*30)
print(f"Використано датчиків: {len(sensor_features)}")
print(f"Список ознак: {'', '.join(sensor_features)}")
print(f"Загальна кількість точок: {len(data_scaled)}")

# Скільки точок прийшло з кожного польоту
for i in range(1, 4):
```

```

count = np.sum(data_cleaned['flight'] == i)
print(f" - Політ №{i}: {count} записів")

n = 15
m = 15
initial_sigma = max(n, m) / 2
initial_lr = 0.5

print("\nПараметри SOM:")
print(f" - Розмір сітки: {n}x{m} ({n*m} нейронів)")
print(f" - Початкова Sigma: {initial_sigma}")
print(f" - Початковий Learning Rate: {initial_lr}")
print("=*30 + "\n")

# 3. Навчання SOM

som = MiniSom(n, m, data_scaled.shape[1],
              sigma=initial_sigma,
              learning_rate=initial_lr,
              random_seed=42)

som.pca_weights_init(data_scaled)
# Кількість ітерацій (для плавного зниження sigma та lr)
iterations = 1000
som.train_batch(data_scaled, iterations, verbose=True)

# 4. Метрики
qe = som.quantization_error(data_scaled)
te = som.topographic_error(data_scaled)

print(f"\nQuantization Error (QE): {qe:.4f}")
print(f"Topographic Error (TE): {te:.4f}")

print("\n" + "=*30)
print(" ІНТЕРПРЕТАЦІЯ РЕЗУЛЬТАТІВ")
print("=*30)

if qe < 3:
    print("Помилка QE: Низька. Нейрони добре підлаштувалися під вектори сенсорів.")
else:
    print("Помилка QE: Помірна. Дані мають високу варіативність (багато шумів).")

if te < 0.1:
    print("Помилка TE: Чудова. Топологія даних повністю збережена.")
elif te < 0.3:
    print("Помилка TE: Хороша. Карта Кохонена адекватно відображає структуру польоту.")
else:
    print("Помилка TE: Висока. Варто збільшити кількість ітерацій.")

# Розрахунок щільності (який відсоток нейронів активовано)
active_neurons = np.sum(som.activation_response(data_scaled) > 0)
print(f"Активация карти: Використано {active_neurons} нейронів з {n*m} можливих.")
print("=*30)

```

```

# 5. Візуалізація

# 5.1. U-Matrix (Unified Distance Matrix)
plt.figure(figsize=(10, 8))
plt.pcolor(som.distance_map().T, cmap='bone_r')
plt.colorbar(label='Відстань')
plt.title('U-matrix (Unified distance matrix)')
plt.show()

# 5.2. Діаграма розсіювання
plt.figure(figsize=(10, 8))
# Фон - U-matrix з низькою прозорістю
plt.pcolor(som.distance_map().T, cmap='bone_r', alpha=0.2)

colors = ['C0', 'C1', 'C2']
labels = ['Політ 1', 'Політ 2', 'Політ 3']

# Отримання координат переможців для кожної точки
w_x, w_y = zip(*[som.winner(d) for d in data_scaled])
w_x = np.array(w_x)
w_y = np.array(w_y)

# Малювання точок з невеликим випадковим зсувом
for i in range(1, 4):
    idx = data_cleaned['flight'] == i
    plt.scatter(w_x[idx] + 0.5 + (np.random.rand(np.sum(idx)) - 0.5) * 0.8,
                w_y[idx] + 0.5 + (np.random.rand(np.sum(idx)) - 0.5) * 0.8,
                s=20, c=colors[i-1], label=labels[i-1], alpha=0.6)

plt.legend()
plt.grid(True, linestyle='--', alpha=0.5)
plt.title('Діаграма розсіювання: Розподіл зразків за вузлами BMU (з Jitter)')
plt.show()

```

Висновок: у ході лабораторної роботи було досліджено застосування самоорганізовних карт Кохонена (SOM) для неконтрольованої кластеризації телеметрії БПЛА DJI Matrice 100. Завдяки попередній обробці даних вдалося навчити модель із високою топологічною точністю ($TE = 0.1896$). Аналіз візуалізацій U-Matrix та діаграм розсіювання підтвердив здатність нейромережі автоматично розрізняти стабільні та нестабільні режими польоту в умовах різного вітрового навантаження на основі восьми параметрів сенсорів.

